

Daily Scholar Papers Report — 2026-09-04

2026-09-04

Daily Scholar Papers Report — 2026-09-04

[Download PDF](#)

Window covered: 2026-09-03 → 2026-09-04 (Google Scholar alerts + user-curated self-emails, last 24 h), extended by the standing 7-day liveness sweep, which this run recovered an entire six-thread alert batch from 1 September that no previous run had seen.

Executive Summary

Ten papers through screening, and what they have in common is a question about **what counts as evidence**. Not what counts as a correct answer — what counts as grounds for believing one.

Four of them, arriving independently from four different research communities, make the same move. Each takes a criterion that was previously a *label* — supplied by a dataset, a benchmark author, or a specification — and replaces it with something the system can check for itself.

ARQ does it for static analysis. CodeQL queries have false positives and false negatives, and the standard way to fix that is to fit the query to a labelled corpus. ARQ instead synthesises C/C++ programs and treats the *disagreement* between what the program does when executed and what the query says about it as the ground truth: silent query plus genuinely vulnerable program means a false-negative weakness; firing query plus safe program means a false-positive one. No labels, no commit history, no vulnerability-specific templates. The refined queries detect up to **119.8 % more true positives at a precision of at least 98.0 %**, close three issues that had been open in the official CodeQL repository for as long as **27 months**, and surface two previously unknown bugs in libpng and zlib.

FuzzingBrain-Bench does it for evaluation. The prevailing way to score an LLM at bug-finding is to ask for a proof-of-concept that triggers one predefined target — which silently discards every valid crash the model finds that happens not to be the one the benchmark author had in mind. The replacement is to count **distinct crash signatures** through a sanitizer-instrumented harness. On 77 challenges from 43 projects, the strongest model evaluated triggers crashes in 60 of them for a score of **196 out of 579**, and **13 challenges defeat every model tested**.

RTL-Obliger does it for specifications, and finds the sharpest number in the window. Give five frontier models a functional RTL spec that, like real specs, simply does not mention its security obligations, and they pass functional tests **73–79 %** of the time and security tests **14–35 %** of the time. The paper's key control is what rules out the obvious explanation: **the stronger a model is functionally, the safer it is not**, and supplying CWE knowledge raises security while unaided self-reflection barely does. The bottleneck is not that the models cannot write defensive hardware. It is that nobody asked them to.

And a survey of **AI slop in vulnerability assessment** states the same thesis at the level of a field. Its framing of hallucinated findings as a cognitive denial-of-service on human triage is the memorable part,

but its argument is the structural one: detection and watermarking address *provenance*, not *correctness*, so the way out is to shift evaluation “from linguistic fluency to mathematical verifiability.”

Against that, the quietest paper in the set is the one worth reading for its method. A **bare-metal firmware sanitization** architecture reports that its program-flow reconstruction handles tail calls, loop functions and multi-threading correctly across all 22 targets — and then spends its remaining pages on where it is still imprecise, tracing the residue to incomplete pointer resolution and unhandled bulk memory operations. It publishes its error budget instead of its win. In a window whose main theme is *what counts as evidence*, that is not a coincidence.

Outstanding: 3 · Keep: 5 · Borderline High-Priority: 2

A note on depth. Retrieval was the binding constraint again. `web_fetch` rate-limited after roughly three documents per cooldown; the arXiv bulk API and direct PDF fetches both returned empty bodies, so the per-paper abstract page was the only working high-yield path. **No paper was read in full this window.** Seven are documented at verified-publisher depth — full abstract, complete author list, licence confirmed at source — and three at Scholar-alert or search-metadata depth. **Every card states its own evidence base**, and no card claims more than its source supports. Two papers are marked Borderline High-Priority not because the work looks marginal but because it could not be retrieved at all; both head the queue for the next run.

Highlighted Papers

Title	Authors	Venue	Link
ARQ: Agentic CodeQL Query Refinement for C/C++ Vulnerability Detection	C. Wang, Y. Ke, J. Yang, Y.-Y. Tsai, P. Li	arXiv preprint, cs.CR / cs.AI, 21 Aug 2026	arXiv:2608.20637
Unsaid, Unsafe? Implicit Security Obligations in LLM-Based RTL Code Generation	G. Yang, X. Hu, X. Chen, X. Xia	arXiv preprint, cs.CR / cs.SE, 27 Aug 2026 (under review)	arXiv:2608.26588
FuzzingBrain-Bench V1: Evaluating Open-Ended Bug Discovery by LLMs	Z. Sheng, A. Kezic, Z. Chen, J. Huang	arXiv preprint, cs.AI / cs.CR / cs.SE, 25 Aug 2026, 21 pp.	arXiv:2608.25158 · benchmark
Answer Is Cheap, Show Me the Evidence! Augmenting Automated Vulnerability Assessment with Evidence	S. Pan, Z. Zheng, J. Zhou, X. Hu, X. Xia, S. Li	ISSTA 2026 (accepted); arXiv preprint 26 Aug 2026	arXiv:2608.25905
AI Slop and Hallucinations in Vulnerability Assessment: A Survey on Reasoning Failures	J. Ding, J. Dong, Y. Zhu, Y. Liu, G. Deng, W. Susilo, S. Ma, Y. Li	SiMLA 2026 (accepted); arXiv preprint 26 Aug 2026	arXiv:2608.25667

Title	Authors	Venue	Link
and Trustworthy Mitigation			
LMSM: LLM Security Framework Inspired by Linux Security Modules	X. Zhang, B. Ruan, J. Fang, A. Zhang, T.-S. Chua, Z. Liang	arXiv preprint, cs.CR, 26 Aug 2026	arXiv:2608.25697
Memory Sanitization for Bare-Metal Firmware Fuzzing via Static Analysis and Memory Access Observation	L. Jungnickel, K. Isakovic, R. Barakat, M. A. Schneider	ARES 2026 — EU Projects Symposium Workshops, LNCS 16903, pp. 662–676	doi:10.1007/978-3-032-37218-5_38
Model-Guided Property-Based Testing of WeChat Pay at Billion-User Scale	X. Shen, Y. Wang, T. Su, J. Liang, J. Sun, X. Liang, H. Sun, X. Xu, H. Lu, Y. Deng, P. Wang, G. Pu, Z. Su, J. Hughes	ASE 2026 — 41st IEEE/ACM Int. Conf. on Automated Software Engineering, Industry Track	author PDF
Partial Cast Calculus for Gradual Information Security	Z. Shen, Z. Xu, S. Qin, Z. Ming	CSF 2026 — 39th IEEE Computer Security Foundations Symposium	IEEE 11662306 · accepted papers
Mitigating Error Propagation in Chain-of-Thought: A Tree-of-Thought Framework for Smart Contract Repair	J. Zhu, H. Wang, X. Li	arXiv preprint, cs.CR / cs.SE, 2026	arXiv:2608.22345

Papers

1.1 · STATIC ANALYSIS · Execution disagreement replaces the labelled dataset, and closes three CodeQL issues open for 27 months [👍😊👉](#)

ARQ: Agentic CodeQL Query Refinement for C/C++ Vulnerability Detection

Authors: Chunyi Wang, Yunfei Ke, Junfeng Yang, Yun-Yun Tsai, Penghui Li

Venue: arXiv preprint, cs.CR (primary) / cs.AI, submitted 21 August 2026. ACM classes D.2.4, I.2.6.

Links. [arXiv:2608.20637](#) · [PDF](#) · [DOI 10.48550/arXiv.2608.20637](#). **Licence:** CC0 1.0 Universal — a public-domain dedication, the most permissive licence in this window. **Evidence base:** verified publisher abstract, full author list and licence. Full text not retrieved (rate limit); all figures and internal numbers below the abstract level are therefore absent.

The gap being targeted

Query-based static analyzers encode vulnerable code patterns as detection queries and match them against source. The queries are hand-written, and like all hand-written patterns they are simultaneously too narrow and too broad: they miss real vulnerabilities and they fire on benign code. Improving them is

normally framed as a supervised problem — fit the query to a labelled corpus of vulnerable and non-vulnerable functions, or mine commit history for fix patterns — which means the quality ceiling of the refined query is the quality of the labels, and every new CWE needs a new template.

The move: let the program adjudicate

ARQ's insight is that a query and a program can be put in direct contradiction, and that the contradiction is self-labelling. Synthesise a C/C++ program. Run it. Ask the query about it. There are exactly two interesting outcomes:

- the program genuinely faults under execution but the query stays silent → the query has a **false-negative weakness**;
- the program is safe under execution but the query fires anyway → the query has a **false-positive weakness**.

Either way the disagreement is a concrete, reproducible defect report about the query, and it comes with the witness program attached. An LLM-based refinement loop then repairs the query using those disagreements as ground truth. The paper is explicit that this requires **no labelled datasets, no commit history, and no vulnerability-specific templates** — the three inputs that every prior query-refinement method depends on.

The economy of this is worth pausing on. The oracle problem for static analysis is usually stated as insurmountable: you cannot decide, in general, whether a flagged path is feasible. ARQ sidesteps rather than solves it by *generating* the programs it reasons about, so that the feasibility question is answered by running them. It buys a sound-enough oracle on a synthetic distribution and spends it on refining an artefact that then generalises to real code.

What the numbers say

Twelve official CodeQL queries were refined using three commercial models (GPT-5.4, Claude-Sonnet-4.6, Gemini-3.5-flash) and evaluated on Juliet v1.3 and FormAI v2 against the original queries:

- **up to 119.8 % more true positives detected** by the refined queries;
- **precision of at least 98.0 % throughout** — the recall gain is not bought with false alarms, which is the failure mode a reader should suspect first;
- **three unresolved GitHub issues** in the official CodeQL query repository fixed, one of which had been open **27 months**;
- **two previously undiscovered bugs** found in libpng and zlib — two of the most scrutinised C libraries in existence.

Why it is worth your time

The libpng and zlib results are the ones that matter. Juliet is synthetic and FormAI is model-generated; both are legitimate targets for the objection that a refined query has learned the benchmark. Finding new bugs in libraries that have been fuzzed continuously for a decade is a different kind of claim, and it is the one that survives the objection.

The transferable idea is not CodeQL-specific and not even LLM-specific: **when you cannot label your data, manufacture data whose labels are a consequence of running it, and use disagreement as your training signal**. That recipe applies to any checker with a decidable dynamic counterpart — taint

specifications against instrumented execution, API-misuse rules against runtime assertions, typestate properties against recorded traces.

Read critically, the open questions are about the synthesis distribution. A generator that produces programs resembling the CWE the query already targets will refine the query toward that neighbourhood; how the synthesised corpus is constructed, and how much of the 119.8 % is attributable to it rather than to the refinement loop, is exactly what the unretrieved full text should be checked for.

1.2 · HARDWARE SECURITY · Functional pass 73–79 %, security pass 14–35 %, and the stronger models are not the safer ones  [🔗](#)

Unsaid, Unsafe? Implicit Security Obligations in LLM-Based RTL Code Generation

Authors: Guang Yang, Xing Hu, Xiang Chen, Xin Xia

Venue: arXiv preprint, cs.CR (primary) / cs.SE, submitted 27 August 2026. Marked “Under Review”.

Links. [arXiv:2608.26588](https://arxiv.org/abs/2608.26588) · [PDF](#) · [DOI 10.48550/arXiv.2608.26588](https://doi.org/10.48550/arXiv.2608.26588). **Licence:** CC BY 4.0. **Evidence base:** verified publisher abstract, full author list and licence. Full text not retrieved (rate limit).

Why hardware makes the argument sharper

Everything written about insecure LLM-generated code has an implicit escape clause: software can be patched. The paper’s opening move is to remove it. RTL that reaches silicon cannot be fixed after the fact, so the security of generated hardware is a terminal property rather than a maintenance problem. That reframing is what earns the empirical study its weight — the same numbers about Python would read as a to-do list.

The benchmark

SECRTL-GEN: 392 tasks grounded in real SoC IP, spanning **five CWE families** and **four hardware description languages** — Verilog, SystemVerilog, VHDL, and Python. Each task carries **black-box functional and security testbenches**, so the two properties are scored independently rather than conflated.

The design decision that makes the benchmark say something is deliberate and stated as such: the functional specifications **intentionally omit the security obligations**, on the grounds that this matches how obligations are actually kept out of functional documentation in industrial practice. The benchmark is therefore not testing whether models can follow security instructions. It is testing what they do when nobody gives them any.

The empirical finding, and the control that makes it interpretable

Across five frontier models under vanilla prompts:

- functional tests pass in roughly **73–79 %** of cases;
- security tests pass in only **14–35 %**.

A gap that size invites a lazy reading — the models are just not good enough yet, and scale will close it. The paper forecloses that reading with a single sentence of evidence: **stronger functional models are not safer**. Functional capability and security capability are decoupled, so waiting for better models is not a plan.

The prompt ablations then localise the cause. Supplying **CWE knowledge raises security**, while **unaided self-thinking helps less** — the model can act on a weakness taxonomy it is given but does not reliably retrieve one on its own. And both security-oriented prompt styles **cut functional pass rates**, which is the uncomfortable half of the result: telling the model to be careful makes it worse at the job. The conclusion the authors draw is precise and is the paper's thesis: the bottleneck is **missing weakness awareness in the specification**, not an inability to write defensive RTL.

The system

RTL-Obliger is neuro-symbolic and each half does what it is good at. An LLM extracts a functional-semantic graph from the specification. A **symbolic engine** matches that graph against a **CWE pattern ontology** to surface mitigation-evidence gaps and signal-level obligations — this is the step that must not hallucinate, and it is the step that is not an LLM. The LLM then revises the RTL under those recovered obligations, in a functionality-preserving two-stage generation, which addresses the ablation's finding that naive security prompting costs functional correctness.

Across five models and four languages, mean all-pass rises from **49.6–51.4 %** for the secure-generation baselines (SecV, RESCUE) to **61.6 %**, with both security and functional rates higher than those baselines.

Why it is worth your time

The ten-point all-pass improvement is respectable, not spectacular; 61.6 % is not a number anyone should tape out on. The paper's value is diagnostic. It isolates a failure that is usually attributed to the model and relocates it to the artefact the model was given — and it does so with the one control that makes the claim falsifiable, by showing capability and safety moving independently.

The generalisation for anyone outside hardware: **your specifications encode obligations that your readers inferred and your generator cannot**. Every requirements document that was written for humans carries this liability, and the remedy demonstrated here — recover the implicit obligations symbolically, then generate under them — is not specific to RTL or to CWEs.

1.3 · FUZZING & BENCHMARKING · Scoring distinct crash signatures instead of one predefined target, and 13 of 77 challenges defeat every model 

FuzzingBrain-Bench V1: Evaluating Open-Ended Bug Discovery by LLMs

Authors: Ze Sheng, Aleksandar Kezic, Zhicheng Chen, Jeff Huang (Ze Sheng and Aleksandar Kezic contributed equally)

Venue: arXiv preprint, cs.AI (primary) / cs.CR / cs.LG / cs.SE, submitted 25 August 2026. 21 pages, 12 figures, 7 tables.

Links. [arXiv:2608.25158](https://arxiv.org/abs/2608.25158) · [PDF](#) · [benchmark corpus and harnesses](#) · [DOI 10.48550/arXiv.2608.25158](https://doi.org/10.48550/arXiv.2608.25158).

Licence: CC BY 4.0. **Evidence base:** verified publisher abstract, full author list and licence. **Full text not retrieved (rate limit).**

The measurement error being corrected

The standard protocol for asking whether a model can find bugs is: here is a project, here is a known vulnerability, produce an input that triggers *it*. The paper's objection is that this scoring rule throws away correct work. A model that finds three real crashes, none of which is the designated one, scores zero.

The measured quantity is therefore not the model's bug-finding ability but its agreement with the benchmark author's choice of target — and those two things diverge exactly where open-ended search is most valuable.

The replacement

Each challenge hands the model an open-source project and a **sanitizer-instrumented harness** inside a **self-contained Docker image**. The objective is restated as: produce inputs that trigger **as many distinct crashes as possible** through that harness. Scoring counts **distinct crash signatures, capped at a predefined maximum** per challenge and **weighted by a difficulty coefficient** — the cap stopping a single shallow crash from being farmed, the weight stopping easy targets from dominating the total.

The sanitizer is doing the real work here, and it is the same structural choice as ARQ's: the oracle is a dynamic, self-checking artefact, not a label. What makes a crash count is that ASan says so.

Composition. 77 challenges from 43 open-source projects: **36 C, 32 C++, 9 Java/JVM**.

What the numbers say

Evaluating Claude Haiku 4.5, Claude Sonnet 4.6 and Claude Opus 4.8 on the full benchmark:

- **Claude Opus 4.8 performs best**, triggering crashes in **60 of 77 challenges**;
- its score is **196 out of 579** — roughly a third of the available points, which is the more informative figure and the one a headline would have omitted;
- **13 challenges are not cracked by any of the three models**.

Why it is worth your time

The 60-of-77 and 196-of-579 numbers tell opposite-sounding stories on purpose, and reporting both is the paper's honesty. Getting a crash somewhere is now routine; getting the crashes that are actually there is not. The headroom, not the pass rate, is the contribution — a benchmark that is 66 % unsolved has somewhere to go, and the 13 untouched challenges are a concrete research agenda rather than a rounding error.

One caveat a reader should hold: the evaluated models are all from one family, so the reported spread is a capability ladder within a single lineage rather than a cross-vendor comparison. The corpus and harnesses are public, which is what makes that fixable by anyone.

1.4 · VULNERABILITY ASSESSMENT · Evidence an analyst can check, not just a label: 5.3–35.2 % over the strongest baseline, ISSTA 2026 

Answer Is Cheap, Show Me the Evidence! Augmenting Automated Vulnerability Assessment with Evidence

Authors: Shengyi Pan, Zelong Zheng, Jiayuan Zhou, Xing Hu, Xin Xia, Shanping Li

Venue: ISSTA 2026 (accepted). arXiv preprint cs.SE, submitted 26 August 2026.

Links. [arXiv:2608.25905](https://arxiv.org/abs/2608.25905) · [PDF](#) · [DOI 10.48550/arXiv.2608.25905](https://doi.org/10.48550/arXiv.2608.25905). **Licence:** CC BY 4.0. **Evidence base:** verified publisher abstract, full author list, venue and licence. Full text not retrieved (rate limit).

The problem with being right

Vulnerability assessment characterises a reported vulnerability so that remediation can be prioritised. Automated methods predict the characterisation from the report. The paper's framing is that prediction accuracy is the wrong target on its own, because the prediction is imperfect and a human analyst must therefore validate it — and a bare label gives that analyst nothing to validate *with*. Hence the title. An answer costs nothing to produce; grounds for accepting it are the scarce good.

There is a second, more mundane gap: existing methods read the report as text and ignore what is actually in it — **screenshots and code snippets** — as well as contextual information about the affected project.

The system

EAVA uses specialised LLM agents to process the rich-text content and the project information, then trains a dedicated assessment model in **two stages**: supervised instruction tuning on **automatically annotated reasoning trajectories** to inject domain knowledge, followed by **reinforcement learning** to improve intrinsic reasoning. At inference it also retrieves **similar historical vulnerabilities** as supplementary evidence — which is the cheapest and most auditable evidence type in the design, since the analyst can read the precedent directly.

What the numbers say

On a newly collected SVR dataset, EAVA **outperforms the strongest baseline by 5.3 to 35.2 % across multiple metrics**. Ablations confirm that both the assessment-specific model training and the information enrichment contribute. A **user study with security experts** reports that the evidence EAVA supplies is useful and practical in real assessment work.

Why it is worth your time

The user study is the part worth borrowing. A claim about explanations that is supported only by an accuracy delta is not a claim about explanations; putting the artefact in front of the people who would have to act on it is the only measurement that matches what the paper says it is optimising. Pairing an offline metric with a practitioner study is a template more evidence-generating systems should copy.

The 5.3–35.2 % range is wide, and the width is itself information — it says the gain is metric-dependent, and the honest reading is that the low end is the one to plan around.

1.5 · VULNERABILITY ASSESSMENT · Hallucinated findings as a denial-of-service on human triage, and why watermarking cannot fix it [!\[\]\(cf531ed27e91483460120fcc057b3901_img.jpg\) !\[\]\(34fde9b7c74442c0438f550a41236260_img.jpg\) !\[\]\(f3ffd03e145adb5d0f6f54d9f4fb82fd_img.jpg\)](#)

AI Slop and Hallucinations in Vulnerability Assessment: A Survey on Reasoning Failures and Trustworthy Mitigation

Authors: Junchen Ding, Jialiang Dong, Yichen Zhu, Yi Liu, Gelei Deng, Willy Susilo, Siqi Ma, Yuekang Li

Venue: SiMLA 2026 (accepted). arXiv preprint cs.CR / cs.AI, submitted 26 August 2026.

Links. [arXiv:2608.25667](https://arxiv.org/abs/2608.25667) · [PDF](#) · [DOI 10.48550/arXiv.2608.25667](https://doi.org/10.48550/arXiv.2608.25667). **Licence:** CC BY 4.0. **Evidence base:** verified publisher abstract, full author list, venue and licence. Full text not retrieved (rate limit).

The framing

Three artefact types are grouped under one name: hallucinated vulnerabilities, plausible-but-incorrect patches, and semantically repackaged bug reports. The survey's contribution to the discourse is the observation that the harm is not primarily *deception* but *volume* — the burden these impose on human triage pipelines is characterised as mirroring a denial-of-service attack. That reframing changes the remedy. You do not defend against a flood by improving your judgement of each item.

The mechanism and its proxy

The root cause identified is the mismatch between the **causal, deductive** reasoning a security analyst performs and the **autoregressive, probabilistic** generation an LLM performs. The survey operationalises that gap as a measurable proxy — the **Deductive Coverage Score** — and reports that chain-of-thought prompting and tool-using agents **narrow but do not close** it. Naming a quantity is what turns an intuition into something a subsequent paper can move.

The argument against the popular fix

The sharpest analytical point is a negative one, and it is aimed at where the field's effort is currently going: **passive detection and watermarking target provenance rather than correctness**, and face fundamental entropy constraints besides. Knowing that a report was machine-written tells you nothing about whether the vulnerability exists. The survey argues instead for **active neuro-symbolic verification**, mapping each pipeline stage onto prior systems with documented limits on security inputs — a mapping that is the most useful thing in a survey for someone deciding what to build.

It closes by specifying two evaluation instruments, **CVE-Bench** and **Slop-Score**, with dataset construction, metric formulas and explicit **anti-gaming provisions**, under the banner of shifting evaluation from linguistic fluency to mathematical verifiability.

Why it is worth your time

Surveys are usually the least methodologically reusable thing in a digest. This one earns its place by taking positions: it says what will not work and why, it defines a metric rather than a taxonomy alone, and it ships proposed benchmarks with anti-gaming clauses attached. Whether the Deductive Coverage Score survives contact with other groups is the open question, and the specification is public enough to be attacked.

1.6 · FIRMWARE SECURITY · A sanitizer where no OS exists, and a paper that spends its last pages on its own error budget 

Memory Sanitization for Bare-Metal Firmware Fuzzing via Static Analysis and Memory Access Observation

Authors: Luca Jungnickel, Karsten Isakovic, Ramon Barakat, Martin A. Schneider (Fraunhofer Institute for Open Communication Systems, FOKUS, Berlin, Germany)

Venue: ARES 2026 — *Availability, Reliability and Security. ARES 2026 EU Projects Symposium Workshops*, Lecture Notes in Computer Science vol. 16903, pp. 662–676. First online 24 August 2026.

Links. [doi:10.1007/978-3-032-37218-5_38](https://doi.org/10.1007/978-3-032-37218-5_38) · [SpringerLink chapter](#). **Licence:** © 2027 The Author(s), under exclusive licence to Springer Nature Switzerland AG — not open access. No figures

reproduced. Funded by Horizon Europe grant agreement No. 101120270 (DOSS). **Evidence base: verified published abstract, full author list, affiliations, licence and reference list from the publisher record. Paywalled full text not retrieved.**

The gap

AddressSanitizer and its relatives are the reason modern fuzzing finds memory-corruption bugs rather than only crashes, and they all assume an operating system underneath — allocator interposition, shadow memory, process abstractions. Bare-metal firmware has none of that. So the most productive part of the fuzzing stack is simply unavailable on the class of targets where memory-corruption bugs are most consequential and hardest to patch.

The architecture

Two components, and the division of labour is the reusable idea:

- **static analysis of the firmware binary**, which recovers the program's structure and the extents of global, stack and heap regions;
- **runtime memory access observation inside the emulator**, which sees every access as it happens.

Built on top of the **Hoedur** firmware fuzzer, and — the constraint that makes it deployable — **without requiring source-code instrumentation**. Firmware binaries are frequently all you have.

What the evaluation reports

On **22 real-world firmware targets**, the architecture correctly reconstructs program flow and intercepts memory accesses that remain invisible to conventional crash- and hang-based fuzzing. Program-flow reconstruction handles **tail calls, loop functions and multi-threaded execution correctly across all targets** — the three control-flow patterns most likely to break a binary-level recovery.

Why it is worth your time

The paper does not claim practical detection accuracy, and says so. It traces the remaining inaccuracies to **incomplete pointer resolution** and **unhandled bulk memory operations**, characterises both as solvable engineering problems rather than fundamental limits, and devotes its closing analysis to the sources of imprecision it encountered.

That is an unusual thing to publish and a valuable one to read. A workshop paper reporting a partially working sanitizer with a careful account of *why* it is partial gives the next group a map; the same work written as a win would give them a number to beat and nothing else. If you work on binary-level analysis, the imprecision taxonomy is the section to read, and it is the section most papers would have cut.

1.7 · LLM SYSTEMS SECURITY · Mechanism/policy separation from LSM applied to serving:
HarmBench ASR 39.20 % → 3.32 % at 98.14 % of throughput 

LMSM: LLM Security Framework Inspired by Linux Security Modules

Authors: XiuYu Zhang, Bonan Ruan, Junfeng Fang, An Zhang, Tat-Seng Chua, Zhenkai Liang

Venue: arXiv preprint, cs.CR, submitted 26 August 2026.

Links. [arXiv:2608.25697](https://arxiv.org/abs/2608.25697) · [PDF](#) · [DOI 10.48550/arXiv.2608.25697](https://doi.org/10.48550/arXiv.2608.25697). **Licence:** CC BY 4.0. **Evidence base:** verified publisher abstract, full author list and licence. Full text not retrieved (rate limit).

The integration problem, which is the actual subject

Interpretability research produces a steady supply of model-internal signals — sparse autoencoder features, transcoders, linear probes — that look like they ought to inform enforcement. The paper's observation is that they are **not security controls by themselves**, and that the way they get deployed makes this worse: each signal arrives coupled to its own calibration, its own policy logic and its own intervention code. Every new interpretability artefact therefore creates integration work instead of strengthening a shared defence. This is a software-architecture complaint, not an ML one, and it is why the paper belongs in an SE reading list.

The design

The borrowed structure is the separation behind **Linux Security Modules**, and the three-way split is stated cleanly:

- a **selected security backend** exposes calibrated evidence;
- a **versioned policy** evaluates active rules over trusted per-request context;
- a **separate gate** authorises buffered output release.

The property this buys is the classical one: **mediation correctness is separated from policy effectiveness**. Whether every relevant operation is intercepted is now a different question, answerable by different means, from whether the rules are any good — and backends, rules or schedules can change without rebuilding request handling or enforcement.

What the prototype shows

Implemented against Hugging Face Transformers and continuously batched **vLLM**. The same substrate hosts artifact-backed **sparse autoencoder** and **transcoder** deployments alongside task-fitted **dense probes**, preserves request-specific decisions under scheduler churn — the hard part under continuous batching — and selectively enforces and composes multiple rules per request.

On **Qwen3-4B**, LMSM-Checkpoint:

- reduces **HarmBench attack success rate from 39.20 % to 3.32 %**;
- raises **XSTest false refusals from 2.40 % to 4.40 %** — the cost, reported rather than omitted;
- retains **98.14 % of the throughput** of a matched serving path doing no monitoring work, at 32 active sequences.

Why it is worth your time

Read this as a reference-monitor paper that happens to have an LLM behind it. The interesting claim is not the HarmBench delta — it is that request-specific security decisions can be preserved under continuous batching at a 1.9 % throughput cost, which is the engineering fact that decides whether any of this ships. The mechanism/policy split is thirty years old and the point is precisely that it transfers; the paper is most useful as a worked example of applying a known systems-security discipline to a stack that currently lacks one.

1.8 · PROPERTY-BASED TESTING · Property-based testing carried into a payment system with over a billion users, with QuickCheck's co-author on the paper [!\[\]\(125d701e9425b54c764340b5671b38cd_img.jpg\) !\[\]\(34c5d6a15de5cee4fef2fa4252527f03_img.jpg\) !\[\]\(5b11d5c5e33a434b0685002e20a1170c_img.jpg\)](#)

Model-Guided Property-Based Testing of WeChat Pay at Billion-User Scale

Authors: Xiangchen Shen, Yiting Wang, Ting Su, Jingjing Liang, Jingling Sun, Xixian Liang, Haiying Sun, Xinjie Xu, Haochuan Lu, Yuetang Deng, Pengcheng Wang, Geguang Pu, Zhendong Su, John Hughes

Venue: ASE 2026 — 41st IEEE/ACM International Conference on Automated Software Engineering, Industry Track.

Links. [author-hosted PDF](#) · [Scholar lookup](#). **Evidence base:** Scholar-alert abstract plus venue and full author list resolved via search. The author-hosted PDF returned an empty body to the fetcher and the full text was not retrieved. Licence undetermined; no figures reproduced. Numbers below are limited to what the alert abstract states.

The setting

Mobile payment correctness is a domain where the cost function is unusually clean: subtle faults produce direct financial loss. The paper reports that the WeChat Pay team — an application with **over one billion active users worldwide** — developed a testing approach in-house, and the paper is the account of it.

Why it is flagged despite thin retrieval

Two things about the author list carry information that the abstract does not. **John Hughes** is a co-author of QuickCheck and the person most associated with property-based testing as a practice; **Ting Su**, **Jingling Sun** and **Zhendong Su** have been building the model-guided and property-based testing line for mobile applications for several years, including the Kea and Kea2 work. An ASE Industry Track paper is the venue where a technique reports what happened when a real organisation adopted it — which is the evidence type that PBT literature has least of.

This also sits directly alongside a paper covered in the [2026-08-30 report](#) on deriving executable properties from natural language for property-based testing of mobile apps, from an overlapping group. Read together, the two describe the same programme from opposite ends: where the properties come from, and what happens when you deploy them at scale.

Status

Borderline High-Priority — flagged, not assessed. No claim is made here about the technique's effectiveness, the size of the deployment, or the defects found, because none of that was retrievable this window. This is first in the deep-read queue for the next run with retrieval budget.

1.9 · PROGRAMMING LANGUAGES · A cast calculus aimed at the known tension between the gradual guarantee and information-flow security [!\[\]\(fe3aebe81acea8d45108cd2768939da7_img.jpg\) !\[\]\(0eef4a60de6ea648e23dfa6079e4dd07_img.jpg\) !\[\]\(9adff8af06744607601d4d78a077407c_img.jpg\)](#)

Partial Cast Calculus for Gradual Information Security

Authors: Zechi Shen, Zhiwu Xu (Shenzhen University), Shengchao Qin, Zhong Ming

Venue: CSF 2026 — 39th IEEE Computer Security Foundations Symposium.

Links. [IEEE Xplore 11662306](#) · [CSF 2026 accepted papers](#). **Evidence base:** Scholar-alert abstract plus venue and affiliations resolved via search. Paywalled; full text not retrieved. Licence undetermined. **No formal characterisation is reproduced below, because none was retrievable and the report does not reconstruct formulas the paper did not supply.**

The tension it addresses

Gradual typing lets a language mix dynamic and static checking and migrate between them, and it has been carried over into information flow control so that security labels can be added incrementally. The property that makes the migration story work is the **gradual guarantee**: removing type annotations should not change runtime behaviour. The alert abstract locates the paper against the result of **Toro et al.**, who identified a problem with holding the gradual guarantee and the security property simultaneously — the point at which gradual security stopped being a straightforward transplant and became a research question.

The contribution, as far as the abstract states it, is a **partial cast calculus** intended to address that difficulty.

Why it is flagged despite thin retrieval

This is the only formal-methods paper in the window and the only one whose core contribution is a metatheoretical result rather than an empirical one. CSF is the venue for exactly this. For a reader whose brief spans PL and security, a calculus that reconciles — or precisely characterises the limits of reconciling — the gradual guarantee with noninterference is a more durable artefact than any of the LLM-pipeline results above it, because it will still be true in five years.

Status

Borderline High-Priority — flagged, not assessed. The specific formulation of the calculus, the statement of what is recovered, and the metatheory are all behind IEEE Xplore. Reporting a formal result second-hand would violate the formula-integrity rule, so nothing further is claimed. Second in the deep-read queue.

1.10 · SMART CONTRACTS · Branch-and-evaluate search substituted for linear chain-of-thought in contract repair [!\[\]\(0b5e7e25e8775f7e7e80906ada4f0021_img.jpg\) !\[\]\(740312fd467f47b04cab841ab3868d83_img.jpg\) !\[\]\(dbb8da2687e90ededffd3484b6b666cf_img.jpg\)](#)

Mitigating Error Propagation in Chain-of-Thought: A Tree-of-Thought Framework for Smart Contract Repair

Authors: Jingping Zhu, Hongping Wang, Xiaoqi Li

Venue: arXiv preprint, cs.CR / cs.SE, 2026.

Links. [arXiv:2608.22345](#) · [PDF](#). **Evidence base:** Scholar-alert abstract and search-resolved author list only. The arXiv abstract page was not retrieved (rate limit), so licence, exact submission date and reported results are undetermined. No numbers are quoted below because none were available at source.

The stated problem

Deployed smart contracts are immutable, so a defect that survives to deployment cannot be patched in place and the financial consequences are immediate. The paper's premise is that current AI-based repair approaches reason **linearly** — a single chain of thought, in which an early mistake conditions everything downstream and there is no mechanism to recover from it.

The stated approach

Replace the linear chain with a **tree-of-thought** structure, so that repair candidates branch and can be evaluated against one another rather than committed to in sequence. The claim is about error propagation: a search over partial solutions can abandon a bad branch, whereas a chain must carry it.

How to read it

The premise — that single-path reasoning compounds early errors — is well established and the structural remedy is a reasonable response to it. What determines whether the approach is worth adopting is entirely in the parts not retrievable this window: what the branch-evaluation criterion is, whether it consults anything external to the model (a compiler, a verifier, a test suite) or only the model's own judgement, how the candidate patches are validated, and what the repair rates are against existing contract-repair baselines.

Note in passing that this is the one paper in the window whose validation strategy could not be established at all — and that in a window where the recurring theme is precisely *which oracle a method trusts*, that is the first question to ask of it.

Status

Keep — documented at alert depth. Worth a look if smart-contract repair is your area; the retrieval gap means nothing here should be taken as an assessment of the results.

Cross-Paper Synthesis

Four papers, four communities, one substitution

Static analysis, hardware generation, fuzzing evaluation, and the vulnerability-triage literature do not usually converge in a single week. This window they did, on a move that is easier to see stated four ways than once.

paper	the criterion it inherited	what it replaced it with
ARQ (1.1)	a labelled corpus of vulnerable functions	execution disagreement between a synthesised program and the query's verdict
FuzzingBrain-Bench (1.3)	"did you hit the predefined target?"	distinct crash signatures reported by a sanitizer
RTL-Obliger (1.2)	the functional specification as written	implicit obligations recovered symbolically from a CWE ontology

paper	the criterion it inherited	what it replaced it with
EAVA (1.4)	a predicted assessment label	evidence an analyst can independently check

In each case the inherited criterion was cheap to state and expensive to trust, and the replacement is a signal the system can produce and verify for itself. ARQ does not need to know which functions are vulnerable; it needs a program that faults. FuzzingBrain-Bench does not need to know which bug the model should have found; it needs ASan to fire. RTL-Obliger does not need the spec to mention CWE-1262; it needs an ontology it can match a semantic graph against. EAVA does not need its prediction to be right; it needs the analyst to be able to tell.

The **AI slop survey** (1.5) is the general statement of what these four are doing operationally, and it names the alternative it rejects: passive detection and watermarking address provenance, not correctness. Its prescription — shift evaluation from linguistic fluency to mathematical verifiability — is the abstract form of all four substitutions above.

Where the theme goes next: the oracle is not free

Read against the [2026-09-01 window](#), whose finding was that the *oracle is the bottleneck*, this one is the constructive follow-up: here is what people are putting in its place. But two papers in this set quietly price the replacement.

Memory sanitization for bare-metal firmware (1.6) is what it costs to have a dynamic oracle at all. The whole architecture — binary static analysis plus emulator-level access observation over Hoedur — exists to manufacture, on a target with no operating system, the thing FuzzingBrain-Bench simply assumes is present in its Docker image. Everyone building on sanitizer-grounded evaluation is standing on infrastructure that does not exist for embedded targets, and that paper is a status report on building it. It is honest about not being finished.

RTL-Obliger (1.2) prices the other half. Its symbolic engine matching a CWE pattern ontology is a curated artefact that somebody has to write and maintain; “recover the implicit obligations” is only cheap where an ontology of obligations already exists. The move from labels to checkable evidence does not eliminate human specification work — it relocates it from per-instance annotation to per-domain infrastructure. That is a much better trade, and it is still a trade.

A note on decoupling, which is the window’s most transferable empirical claim

The single most useful sentence in this set is RTL-Obliger’s finding that **stronger functional models are not safer**. It is a decoupling result, and decoupling results are what let you stop waiting.

It rhymes with 1.5’s report that chain-of-thought prompting and tool use *narrow but do not close* the deductive-reasoning gap, and with 1.7’s throughput measurement showing that enforcement need not cost capability — 98.14 % of baseline throughput. Three papers, three axes, same shape: the quantity everyone assumes is coupled to general capability turns out to move on its own. Where that is true, architecture beats scale, and the fix is available now rather than at the next model release.

On honest reporting, which showed up more than once

Three papers in this window report the number that undercuts them, and it is worth naming the pattern because it is a choice each of them made. FuzzingBrain-Bench pairs “60 of 77 challenges” with “196 of

579 points” and “13 challenges nobody solved”. LMSM pairs a 91 %-relative reduction in attack success with a near-doubling of false refusals. The firmware sanitization paper leads with what works across all 22 targets and closes with a taxonomy of where it does not. In each case the deflating number is more useful to a reader than the headline, and in each case a less careful version of the paper would have had a stronger abstract and less value.

Writing & Rationale Insights

Remove the escape clause before you present the numbers. RTL-Obliger (1.2) opens by observing that insecure software can be patched after deployment and insecure silicon cannot. Everything after that sentence lands harder, because the reader has been deprived of the standard consolation. If your setting has a property that makes the problem terminal rather than annoying, establish it in the first paragraph — it converts the same measurements from a to-do list into a finding.

Design the flaw into the benchmark on purpose, and say you did. SECRTL-GEN’s functional specifications *intentionally* omit the security obligations, justified by how obligations are actually kept out of functional docs in practice. Stating the omission as a deliberate design decision with an industrial rationale converts what a reviewer would otherwise call an unfair setup into the entire point of the instrument.

Publish the control that kills the boring explanation. The 73–79 % versus 14–35 % gap invites “the models aren’t good enough yet.” One clause — stronger functional models are not safer — makes that reading untenable. Identify the lazy interpretation of your headline result and spend a sentence of evidence foreclosing it; it is the cheapest credibility available.

Report the number that makes your result look smaller. FuzzingBrain-Bench (1.3) could have written “60 of 77 challenges cracked.” It also reports 196 of 579 points and 13 challenges no model touched. The second pair is what makes the benchmark a research agenda instead of a scoreboard, and volunteering it is what makes the first number believable.

State the cost of your defence in the same breath as its benefit. LMSM (1.7) reports the false-refusal rise from 2.40 % to 4.40 % alongside the attack-success drop from 39.20 % to 3.32 %, and the throughput retention alongside both. A security result with no reported cost invites the reader to assume the cost was not measured.

Spend your last section on your error budget. The firmware sanitization paper (1.6) closes by systematically analysing its own sources of imprecision and naming them — incomplete pointer resolution, unhandled bulk memory operations — as solvable engineering problems. For a workshop paper reporting an unfinished system, this is the section that makes it citable: it hands the next group a map instead of a number to beat.

Name your gap so someone else can move it. The AI-slop survey (1.5) does not stop at describing the mismatch between deductive and probabilistic reasoning; it defines the Deductive Coverage Score as a proxy and ships two evaluation instruments with anti-gaming provisions. A survey that only classifies is read once. A survey that leaves behind a measurable quantity gets built on.

Let the artefact be judged by the people who would use it. EAVA (1.4) supports its claim about evidence quality with a user study of security experts rather than an accuracy delta alone. If your

contribution is framed as helping a human decide, the accuracy metric is not measuring your contribution.

Argue against the popular remedy explicitly. 1.5 states that watermarking and passive detection target provenance rather than correctness. Naming where the field's current effort is misdirected — and saying why, in terms of what the technique can and cannot establish — is more valuable than adding one more method to the pile, and it is the part of a survey a reader will still be quoting a year later.